Get_azure_role_assignments-at subscription only level

Friday, December 1, 2023 11:39 AM

```
# Import required modules
Import-Module Az.Accounts
Import-Module Az.Resources


Set-Item Env:\SuppressAzurePowerShellBreakingChangeWarnings 'true'

Connect-AzAccount #-identity

$date = get-date
$Azrolesreport = ''

$subscriptions =  Get-AzSubscription
$i = 0

foreach($sub in $subscriptions)
{
   $subscriptionName = $sub.name
   $token = Get-AzAccessToken
   set-AZcontext -subscription $subscriptionname


      $i = $i +1
      write-host " $($sub.name)  - $($subscriptions.count -$i)" -ForegroundColor Green

      $allassignments = Get-AzRoleAssignment -Scope "/subscriptions/$($sub.id)"

      foreach($assignment in $allassignments)
      {
        if($($assignment.Scope) -eq "/subscriptions/$($sub.id)" )
        {
           $inheritancesource = "made directly on the resource $($resource.Name)"
           $inheritance = 'no direct assignment'
           write-host "$inheritance" -ForegroundColor cyan
        }
        else
        {
           $inheritancesource = "inherited from scope $($assignment.Scope)"
           $inheritance = 'yes'
        }
        try
          {
             # Attempt to get the user
             $user = Get-AzADUser -ObjectId $($assignment.objectid)  -erroraction ignore

             if($user -ne $null)
             {
                # Print the user, resource, and role details
                #  Write-Output "User: $($user.DisplayName) Resource: $($resource.Name) Role:
$($roleDefinition.Name) Role Description: $($roleDefinition.Description)"
                $username = $($user.DisplayName)
             }
          }
          catch
          {
             # If the user retrieval failed, it might be a group
             try
             {
                # Attempt to get the group
                $group = Get-AzADGroup -ObjectId $objectId

                if($group -ne $null)
                {
                   # Print the group, resource, and role details
                    # Write-Output "Group: $($group.DisplayName) Resource: $($resource.Name) Role:
$($roleDefinition.Name) Role Description: $($roleDefinition.Description)"
                   $groupname = $($group.DisplayName)
                }
             }
             catch
             {
                # If the group retrieval also failed, it might be a service principal or something else
                #Write-Output "Unrecognized ObjectId: $objectId Resource: $($resource.Name) Role:
$($roleDefinition.Name) Role Description: $($roleDefinition.Description)"
                $groupname = 'none'
             }
          }



        $roleobj = new-object PSObject


        $roleobj | Add-Member -MemberType NoteProperty -name RoleAssignmentName -value
$($assignment.DisplayName)
        $roleobj | Add-Member -MemberType NoteProperty -name RoleAssignmentID -value
$($assignment.RoleAssignmentId)
        $roleobj | Add-Member -MemberType NoteProperty -name inheritance -value $inheritance
        $roleobj | Add-Member -MemberType NoteProperty -name user -value $username
        $roleobj | Add-Member -MemberType NoteProperty -name Group -value $groupname
        $roleobj | Add-Member -MemberType NoteProperty -name inheritancesource -value
$inheritancesource
        $roleobj | Add-Member -MemberType NoteProperty -name DisplayName -value
$($assignment.DisplayName)
        $roleobj | Add-Member -MemberType NoteProperty -name SignInName -value
$($assignment.SignInName)
        $roleobj | Add-Member -MemberType NoteProperty -name RoleDefinitionName -value
$($assignment.RoleDefinitionName)
        $roleobj | Add-Member -MemberType NoteProperty -name Subscriptionname -value
$($sub.name)
        $roleobj | Add-Member -MemberType NoteProperty -name Subscriptionid -value $($sub.Id)
        $roleobj | Add-Member -MemberType NoteProperty -name ObjectType -value
$($assignment.ObjectType)
        $roleobj | Add-Member -MemberType NoteProperty -name CanDelegate -value
$($assignment.CanDelegate)
        $roleobj | Add-Member -MemberType NoteProperty -name Scope -value $($assignment.Scope)



        [array]$Azrolesreport += $roleobj
      }

     Write-Progress -Activity "Getting role assignments for $resource" -PercentComplete
($resources.IndexOf($resource) / $resources.Count * 100)

}

 ###GENERATE HTML Output for review

   $CSS = @"
  Azure Role Audit $date
<Title> Azure Role Audit $date Report: $date </Title>
```

```
<Style>
th {
        font: bold 11px "Trebuchet MS", Verdana, Arial, Helvetica,
        sans-serif;
        color: #FFFFFF;
        border-right: 1px solid #4B0082;
        border-bottom: 1px solid #4B0082;
        border-top: 1px solid #4B0082;
        letter-spacing: 2px;
        text-transform: uppercase;
        text-align: left;
        padding: 6px 6px 6px 12px;
        background: #5F9EA0;
}
td {
        font: 11px "Trebuchet MS", Verdana, Arial, Helvetica,
        sans-serif;
        border-right: 1px solid #4B0082;
        border-bottom: 1px solid #4B0082;
        background: #fff;
        padding: 6px 6px 6px 12px;
        color: #4B0082;
}
</Style>
"@




 (($Azrolesreport| SELECT  `
    RoleAssignmentName `
    ,RoleAssignmentID `
    ,inheritance `
    ,inheritancesource `
    ,displayname `
    ,SignInName `
    ,RoleDefinitionName `
    ,Subscriptionname `
    ,Subscriptionid `
    ,ObjectType `
    ,CanDelegate `
    ,Scope  | `
ConvertTo-Html -Head $CSS ).replace("root","<font color=red>
root</font>")).replace("subscriptions","<font color=green>subscriptions</font>"))| out-file "C:\TEMP
\azure_sub_role_audit.html"
Invoke-Item   "C:\TEMP\azure_sub_role_audit.html"


######## Prep for export to storage account

$resultsfilename = 'rolessubauditreport.csv'

$rolesauditreport =  $Azrolesreport| SELECT  `
      RoleAssignmentName `
    ,RoleAssignmentID `
    ,inheritance `
    ,inheritancesource `
    ,displayname `
    ,SignInName `
    ,RoleDefinitionName `
    ,Subscriptionname `
    ,Subscriptionid `
    ,ObjectType `
    ,CanDelegate `
    ,Scope  | export-csv  $resultsfilename  -notypeinformation



 ##### storage sub info and creation

#connect-azaccount ## only uncomment if using a storage account under another tenant or account for
consilidation of reports



##########################################################################################
#####################################
 ###

 #### Change subscription , Region, Resourcegroupname, Storageaccountname below

$Region =  "West US"   ## pick storage account region

 $subscriptionselected = 'wolffentpSub'   ### designated storage account subscription if different from
current running subscription



$resourcegroupname = 'wolffautomationrg'
$subscriptioninfo = get-azsubscription -SubscriptionName $subscriptionselected
$TenantID = $subscriptioninfo | Select-Object tenantid
$storageaccountname = 'wolffautosa'   ## dedicate storage account
$storagecontainer = 'rolesaudit'   ### Container for export


### end storagesub info

set-azcontext -Subscription $($subscriptioninfo.Name)  -Tenant $($TenantID.TenantId)



#BEGIN Create Storage Accounts



 try
 {
    if (!(Get-AzStorageAccount -ResourceGroupName $resourcegroupname -Name
$storageaccountname ))
    {
       Write-Host "Storage Account Does Not Exist, Creating Storage Account: $storageAccount Now"

       # b. Provision storage account
       New-AzStorageAccount -ResourceGroupName $resourcegroupname  -Name
$storageaccountname -Location $region -AccessTier Hot -SkuName Standard_LRS -Kind BlobStorage -
Tag @{"owner" = "Jerry wolff"; "purpose" = "Az Automation storage write" } -Verbose


       Get-AzStorageAccount -Name   $storageaccountname  -ResourceGroupName
$resourcegroupname  -verbose
    }
  }
  Catch
  {
       WRITE-DEBUG "Storage Account Aleady Exists, SKipping Creation of $storageAccount"

  }
       $StorageKey = (Get-AzStorageAccountKey -ResourceGroupName $resourcegroupname  –
```

```
StorageAccountName $storageaccountname).value | select -first 1
      $destContext = New-azStorageContext  –StorageAccountName $storageaccountname `
                        -StorageAccountKey $StorageKey


         #Upload  .csv to storage account

      try
        {
          if (!(get-azstoragecontainer -Name $storagecontainer -Context $destContext))
            {
              New-azStorageContainer $storagecontainer -Context $destContext
            }
        }
      catch
        {
          Write-Warning " $storagecontainer container already exists"
        }


       Set-azStorageBlobContent -Container $storagecontainer -Blob $resultsfilename  -File
$resultsfilename -Context $destContext -Force
```

Summary

- The script imports two modules: **Az.Accounts** and **Az.Resources**, which are used to manage Azure accounts and resources respectively.
- The script sets an environment variable to suppress warnings about breaking changes in Azure PowerShell.
- The script connects to Azure using the default identity of the current user or service principal.
- The script gets the current date and initializes an empty variable for the role assignments report.
- The script loops through all the subscriptions that the current user or service principal has access to, and sets the context to each subscription.
- For each subscription, the script gets an access token and prints the subscription name and the remaining count.
- The script gets all the role assignments for the current subscription scope, and loops through them.
- For each role assignment, the script checks if the scope is the same as the subscription scope, and sets the inheritance source and inheritance flag accordingly.
- The script tries to get the user or group object associated with the role assignment object ID, and handles any errors if the object is not found or is a service principal or something else.
- The script creates a custom object with the properties of the role assignment, such as name, ID, inheritance, user, group, inheritance source, display name, sign in name, and role definition name.
- The script adds the custom object to the role assignments report variable.

The script is finished. The role assignments report variable contains the custom objects with the details of the role assignments for each subscription. You can export the report to a CSV file or display it on the console